

Unity FPS Damage, Barrels, and Door/Key Guide

Unity FPS Damage System, Explosive Barrels, and Door/Key Guide

This guide builds on the working Pick and Drop system, adding: player health with a screen damage flash, explosive/destructible barrels, and a key + locked door interaction system. It reflects the actual working setup, including fixes for issues hit along the way (script/class name mismatches, UI anchor stretching, and using `GetComponentInParent` so explosions correctly detect the player).

PART 1: PLAYER DAMAGE FLASH

Step 1: Create the Damage Flash UI

1. In the Hierarchy: right-click -> **UI -> Canvas** (skip if one already exists).
2. Right-click the Canvas -> **UI -> Image**. Rename this child object `DamageFlash`.
3. Select `DamageFlash`. In the Rect Transform panel, click the **Anchor Presets** icon (top-left box icon), then **hold Shift + Alt** and click the bottom-right **stretch/stretch** option. This makes the image fill the entire screen.
4. After stretching, confirm:
 - **Rotation**: X, Y, Z all `0`
 - **Scale**: X, Y, Z all `1`
 - **Left / Top / Right / Bottom**: all `0`
5. On the **Image** component:
 - Click the circle icon next to **Source Image** and choose **UISprite** (Unity's built-in solid sprite) — without a sprite assigned, the color will not render.
 - Click the **Color** swatch, set it to red, and set **Alpha (A)** to `0` so it starts invisible.

Step 2: Create `PlayersHealth.cs`

Create a new C# script named exactly `PlayersHealth` (class name must match filename exactly).

```
using UnityEngine;
using UnityEngine.UI;

public class PlayersHealth : MonoBehaviour
{
    public int maxHealth = 100;
    public int currentHealth;
    public Image damageFlashImage;
    public float flashDuration = 0.3f;

    private float flashTimer;

    void Start()
    {
        currentHealth = maxHealth;
    }

    void Update()
    {
        if (flashTimer > 0)
        {
            flashTimer -= Time.deltaTime;
            float alpha = Mathf.Clamp01(flashTimer / flashDuration) * 0.5f;
            SetFlashAlpha(alpha);
        }
    }

    public void TakeDamage(int amount)
    {

```

```

        currentHealth -= amount;
        currentHealth = Mathf.Clamp(currentHealth, 0, maxHealth);
        flashTimer = flashDuration;

        Debug.Log("Player took " + amount + " damage. Health: " + currentHealth);

        if (currentHealth <= 0)
        {
            Die();
        }
    }

    void SetFlashAlpha(float alpha)
    {
        if (damageFlashImage != null)
        {
            Color c = damageFlashImage.color;
            c.a = alpha;
            damageFlashImage.color = c;
        }
    }

    void Die()
    {
        Debug.Log("Player died");
    }
}

```

Step 3: Attach and wire it up

1. Select the **Camera** GameObject (the actual rendering camera, same object that already has `pickanddrop` attached).
2. **Add Component -> PlayersHealth**.
3. Drag the `DamageFlash` Image (from the Canvas) into the **Damage Flash Image** field.

PART 2: EXPLOSIVE BARRELS

Step 1: Create `ExplosiveBarrel.cs`

```

using UnityEngine;

public class ExplosiveBarrel : MonoBehaviour
{
    public int health = 50;
    public int explosionDamage = 40;
    public float explosionRadius = 5f;
    public GameObject explosionEffect;

    public void TakeDamage(int amount)
    {
        health -= amount;
        if (health <= 0)
        {
            Explode();
        }
    }

    void Explode()
    {
        if (explosionEffect != null)
        {
            Instantiate(explosionEffect, transform.position, Quaternion.identity);
        }
    }
}

```

```

    }

    Collider[] hits = Physics.OverlapSphere(transform.position, explosionRadius);
    foreach (Collider hit in hits)
    {
        PlayersHealth player = hit.GetComponentInParent<PlayersHealth>();
        if (player != null)
        {
            player.TakeDamage(explosionDamage);
        }

        Rigidbody rb = hit.GetComponent<Rigidbody>();
        if (rb != null)
        {
            Vector3 dir = (hit.transform.position - transform.position).normalized;
            rb.AddForce(dir * 500f);
        }
    }

    Destroy(gameObject);
}
}

```

Important: `GetComponentInParent<PlayersHealth>()` is used instead of `GetComponent<PlayersHealth>()`. Since `PlayersHealth` lives on the Camera (which typically has no collider), a plain `GetComponent` on the hit collider would never find it. `GetComponentInParent` searches up the hierarchy from whatever collider was hit, which is far more reliable for FPS rigs.

Step 2: Attach to each barrel

For each barrel object (e.g. `P_LPSP_Explosive_Barrel_1`, `_2`, `_3`, `_4`):

1. Select it.
2. Confirm it already has **Rigidbody** and a **Collider** (from the pick-and-drop setup).
3. **Add Component -> ExplosiveBarrel.**
4. Optionally assign an **Explosion Effect** prefab (search `Assets -> Infima Games -> Low Poly Shooter Pack -> Prefabs -> Effects` for an existing explosion VFX prefab) for a visible explosion.

Step 3: Add a test trigger (temporary, for verifying it works)

In `pickanddrop.cs`, inside `Update()`, add:

```

if (Keyboard.current.fKey.wasPressedThisFrame)
{
    TryDamageBarrel();
}

```

And add this new method to the same class:

```

void TryDamageBarrel()
{
    Ray ray = Camera.main.ScreenPointToRay(new Vector3(Screen.width / 2, Screen.height / 2, 0));
    RaycastHit hit;

    if (Physics.Raycast(ray, out hit, pickupDistance))
    {
        ExplosiveBarrel barrel = hit.collider.GetComponent<ExplosiveBarrel>();
        if (barrel != null)
        {
            barrel.TakeDamage(25);
        }
    }
}

```

Later, replace this test call with a call from your actual weapon's bullet-hit logic: `barrel.TakeDamage(bulletDamage);`

Step 4: Test

1. Press Play, aim at a barrel, press **F** twice (health 50, 25 damage per hit).
2. The barrel should disappear (destroyed on explosion).
3. If you are standing within `explosionRadius`, the screen should flash red and the Console should log "Player took X damage...".

PART 3: DOOR + KEY SYSTEM

Step 1: Create `PlayerInventory.cs`

```
using UnityEngine;
using System.Collections.Generic;

public class PlayerInventory : MonoBehaviour
{
    private HashSet<string> collectedKeys = new HashSet<string>();

    public void CollectKey(string keyID)
    {
        collectedKeys.Add(keyID);
        Debug.Log("Collected key: " + keyID);
    }

    public bool HasKey(string keyID)
    {
        return collectedKeys.Contains(keyID);
    }
}
```

Attach this to the same **Camera** object as `pickanddrop` and `PlayersHealth`.

Step 2: Create `KeyItem.cs`

```
using UnityEngine;

public class KeyItem : MonoBehaviour
{
    public string keyID = "RedKey";
}
```

Create a key object in the scene:

1. **GameObject** -> **3D Object** -> **Cube**, rename it `Key_Red`, scale it down.
2. Add a **Box Collider** (Is Trigger unchecked).
3. **Add Component** -> **KeyItem**, set **Key ID** to `RedKey`.

Step 3: Create `DoorController.cs`

```
using UnityEngine;
using System.Collections;

public class DoorController : MonoBehaviour
{
    public string requiredKeyID = "RedKey";
}
```

```

public float openAngle = 90f;
public float openSpeed = 2f;
private bool isOpen = false;
private Quaternion closedRotation;
private Quaternion openRotation;

void Start()
{
    closedRotation = transform.rotation;
    openRotation = closedRotation * Quaternion.Euler(0, openAngle, 0);
}

public void TryOpen(PlayerInventory inventory)
{
    if (isOpen) return;

    if (inventory.HasKey(requiredKeyID))
    {
        StartCoroutine(OpenDoor());
    }
    else
    {
        Debug.Log("Door is locked. Requires: " + requiredKeyID);
    }
}

IEnumerator OpenDoor()
{
    {
        isOpen = true;
        float t = 0;
        while (t < 1f)
        {
            t += Time.deltaTime * openSpeed;
            transform.rotation = Quaternion.Slerp(closedRotation, openRotation, t);
            yield return null;
        }
    }
}
}

```

Door object setup:

1. Create or import a door model — a stretched Cube works for testing (e.g. scale 0.2, 3, 1.5).
2. Position it so its pivot sits at one edge, like a hinge. If using a plain cube, create an empty parent GameObject at the hinge position and make the cube a child offset to one side, so rotation happens around the hinge.
3. Add a **Box Collider** to the door.
4. **Add Component -> DoorController**, set **Required Key ID** to `RedKey` (must match the key's ID exactly).

Step 4: Wire key and door interaction into `pickanddrop.cs`

Update `pickanddrop.cs` so pressing E also handles doors and keys, not just `Pickupable` objects:

```

using UnityEngine;
using UnityEngine.InputSystem;

public class pickanddrop : MonoBehaviour
{
    public Transform holdPoint;
    private GameObject heldObject;
    private float pickUpDistance = 3f;
    private PlayerInventory inventory;

    void Start()
    {
        inventory = GetComponent<PlayerInventory>();
    }
}

```

```

void Update()
{
    if (Keyboard.current == null) return;

    if (Keyboard.current.eKey.wasPressedThisFrame && heldObject == null)
    {
        TryInteract();
    }
    else if (Keyboard.current.qKey.wasPressedThisFrame && heldObject != null)
    {
        Drop();
    }

    if (Keyboard.current.fKey.wasPressedThisFrame)
    {
        TryDamageBarrel();
    }
}

void TryInteract()
{
    Ray ray = Camera.main.ScreenPointToRay(new Vector3(Screen.width / 2, Screen.height / 2, 0));
    RaycastHit hit;

    if (Physics.Raycast(ray, out hit, pickupDistance))
    {
        DoorController door = hit.collider.GetComponent<DoorController>();
        if (door != null)
        {
            door.TryOpen(inventory);
            return;
        }

        KeyItem key = hit.collider.GetComponent<KeyItem>();
        if (key != null)
        {
            inventory.CollectKey(key.keyID);
            Destroy(key.gameObject);
            return;
        }

        Pickupable pickupable = hit.collider.GetComponent<Pickupable>();
        if (pickupable != null)
        {
            heldObject = hit.collider.gameObject;
            Rigidbody rb = heldObject.GetComponent<Rigidbody>();
            rb.useGravity = false;
            rb.isKinematic = true;
            heldObject.transform.SetParent(holdPoint);
            heldObject.transform.localPosition = Vector3.zero;
            heldObject.transform.localRotation = Quaternion.identity;
        }
    }
}

void Drop()
{
    Rigidbody rb = heldObject.GetComponent<Rigidbody>();
    heldObject.transform.SetParent(null);
    rb.useGravity = true;
    rb.isKinematic = false;
    heldObject = null;
}

void TryDamageBarrel()
{
    Ray ray = Camera.main.ScreenPointToRay(new Vector3(Screen.width / 2, Screen.height / 2, 0));
    RaycastHit hit;

```

```

        if (Physics.Raycast(ray, out hit, pickUpDistance))
        {
            ExplosiveBarrel barrel = hit.collider.GetComponent<ExplosiveBarrel>();
            if (barrel != null)
            {
                barrel.TakeDamage(25);
            }
        }
    }
}

```

Step 5: Test

1. Press Play. Walk to `Key_Red`, aim at it, press **E**. It should disappear and the Console should log `"Collected key: RedKey"`.
2. Walk to the door, aim at it, press **E**. It should smoothly rotate open since you now hold the key.
3. To confirm the locked case, remove the key pickup step temporarily and try opening the door directly — Console should log `"Door is locked. Requires: RedKey"`.

TROUBLESHOOTING NOTES (from real debugging on this project)

- **"Tag: X is not defined" when using `CompareTag`**: happened even when the tag was visibly listed in *Edit -> Project Settings -> Tags and Layers*, especially on prefab instances with tag overrides. Solved by using component-based markers (`Pickupable`) instead of string tags.
- **E key not registering at all**: caused by *Active Input Handling* being set to only the new Input System, which silently breaks legacy `Input.GetKeyDown()` calls. Fixed by setting it to **"Both"** in *Edit -> Project Settings -> Player -> Other Settings*, and by using `Keyboard.current` from `UnityEngine.InputSystem` directly in scripts.
- **Script fields not showing in the Inspector (e.g. "Hold Point" missing)**: caused by the class name inside the script not matching the filename exactly, or missing `: MonoBehaviour`. Unity requires an exact match.
- **Explosion doesn't damage the player**: `PlayersHealth` lives on the Camera, which usually has no collider, so `GetComponent<PlayersHealth>()` on a hit collider fails silently. Fixed by using `GetComponentInParent<PlayersHealth>()` instead.
- **UI damage flash not covering the screen**: caused by the Image's Rect Transform being anchored to a single center point (0.5, 0.5) instead of stretched. Fixed via Anchor Presets, Shift+Alt+click the stretch/stretch option.
- **UI Image invisible even with correct alpha**: caused by no Sprite assigned to **Source Image**. Assigning Unity's built-in `UISprite` fixed it.
- **Mixing up which code belongs in which file**: `Explode()` and `explosionDamage` belong in `ExplosiveBarrel.cs`, not `PlayersHealth.cs`. Keep each script focused on a single responsibility to avoid this.